Fall Semester 2026-27

BITE305L - Computer Networks

PROJECT REPORT

RAPTOR: PROTOCOL-AWARE IDEMPOTENCY GUARD PROXY AND FAULT-INJECTION HARNESS FOR RESILIENT AGENT-TOOL APPLICATION-LAYER NETWORKS UNDER PACKET LOSS AND TRANSPORT DELAYS

Submitted by

ATHARVA MENDHULKAR

(Register No: 24BIT0103)

AAKASH SEHGAL

(Register No: 24BIT0091)

Under the Guidance of

Dr. SANTHI K

Associate Professor

VIT, Vellore.

DECLARATION

I hereby declare that the project report titled **"RAPTOR: Protocol-Aware Idempotency Guard Proxy and Fault-Injection Harness for Resilient Agent-Tool Application-Layer Networks under Packet Loss and Transport Delays"** submitted in partial fulfilment of the requirements for the course **BITE305L - Computer Networks** is a record of original work carried out by me under the guidance of **Dr. Santhi K**, Associate Professor, Vellore Institute of Technology, Vellore.

I affirm that this work has been developed independently and has not been submitted, either in part or in full, to any other course, institution, or university for the award of any degree, diploma, or certification.

I further declare that:

- All sources of information utilised in this project have been properly acknowledged and cited in accordance with academic standards.
- The work does not contain any form of plagiarism beyond permissible academic limits.
- The network analysis, protocol design, fault-injection harness implementation, proxy middleware, and experimental benchmarks presented in this report are my own contributions, except where explicitly stated.
- Any technical assistance received during the course of this work has been duly acknowledged.
- I am submitting this project as part of my curricular course requirements under the supervision of Dr. Santhi K.

TABLE OF CONTENTS

S.No.	Contents	Page No.
1	Abstract	iv
2	**Chapter 1: Introduction**	**1**
	1.1 Motivation	1
	1.2 Scope and System Boundaries	2
	1.3 Problem Statement & The Two Generals Problem in Tool Networks	2
	1.4 Engineering Objectives	3
	1.5 Organisation of the Report	4
3	**Chapter 2: Literature Survey**	**5**
	2.1 Overview of Network Reliability & Transport Idempotency	5
	2.2 Review of Existing Network Reliability Methods	5
	2.3 Comparative Analysis of Prior Art (Table 2.1)	7
	2.4 Research Gap and Positioning	7
4	**Chapter 3: Proposed Technique**	**9**
	3.1 Network Architecture & 4-Tier Middlebox Topology (Figure 3.1 & 3.2)	9
	3.2 Module-wise Description	10
	3.2.1 Application-Layer Protocol Engine (Model Context Protocol / SSE)	11
	3.2.2 Wire-Level Network Fault Injector (Toxiproxy & ASGIFaultProxy)	11
	3.2.3 Linux Traffic Control () Network Emulation Container	11
	3.2.4 Deterministic Semantic Key Derivation Engine	12
	3.2.5 In-Flight Singleflight Request Coalescer Subsystem	12
	3.2.6 Durable WAL State Ledger & Crash Recovery	13
	3.2.7 Protocol-Aware Reverse Guard Proxy Middleware	13
	3.2.8 In-Process SDK Subsystem	13
	3.2.9 Automated Benchmark Matrix Runner Subsystem	14
	3.2.10 Interactive Web Dashboard & Real-Time Monitoring Console	14

	3.3 Algorithms and Formal Reasoning	14
	3.3.1 Algorithm 3.1: Protocol-Aware Idempotency Verification & Replay	14
	3.3.2 Algorithm 3.2: Non-Blocking Singleflight Network Flow Coalescing	16
	3.3.3 Formal State Transition Lattice & Crash Consistency (Table 3.2)	16
	3.3.4 Mathematical Proof of Zero-Multiplicity Invariant under Arbitrary Packet Loss	17
	3.3.5 Resolution of Open Technical Questions (OTQ Matrix, Table 3.4)	18
	3.4 Network Packet Schema & Event Data Model (Table 3.1)	19
	3.5 Tools and Technologies (Table 3.5)	19
5	**Chapter 4: Results and Discussion**	**21**
	4.1 Experimental Setup & Network Topology	21
	4.2 Empirical Demonstrations and Case Studies	21
	4.2.1 Live System Demonstration: Unprotected Stack Failure under Fault F1	21
	4.2.2 Live System Demonstration: RAPTOR Guard Proxy Protection under Fault F1	22
	4.2.3 Terminal Output: In-Process SDK Deduplication Execution	23
	4.2.4 Terminal Output: Full Matrix Benchmark Run across Adapters & Faults	23
	4.2.5 Web Dashboard: Real-Time Matrix Benchmark Overview	24
	4.2.6 Web Dashboard: Guard Ledger & Packet Event Inspector	24
	4.2.7 Terminal Output: YAML Scenario Deterministic Fault Injection & Replay	25
	4.2.8 Terminal Output: Linux Traffic Control Emulation	26
	4.3 Performance Metrics & Benchmarks (Tables 4.1 & 4.2, Figure 4.9)	26
	4.3.1 Network Round-Trip Time (RTT) & Guard Proxy Latency Overhead	27
	4.3.2 Singleflight Concurrency Scaling & Load Absorption	27
	4.3.3 Multiplicity Elimination across Network Fault Grid	28
	4.4 Automated Verification Suite & Network Invariant Analysis	29
6	**Chapter 5: Conclusion and Future Work**	**31**
	5.1 Conclusion	31
	5.2 Future Work	31
7	**References**	**33**

ABSTRACT

In modern distributed computing and autonomous agent systems, client applications orchestrate critical business operations (e.g., electronic fund transfers, cloud infrastructure provisioning, and transactional database modifications) over packet-switched IP networks using emerging application-layer Remote Procedure Call (RPC) protocols such as the Model Context Protocol (MCP 2024-11-05), JSON-RPC 2.0, and Server-Sent Events (SSE) over Streamable HTTP. However, real-world wide-area and local-area networks are fundamentally unreliable: packet loss, downstream connection drops, latency jitter, and socket timeouts routinely disrupt in-flight transport connections.

Under the classical **Two Generals' Problem**, a client cannot distinguish whether a dropped TCP connection or socket read timeout occurred **before** the server processed the request or **after** the server committed the side effect but **before** the response packet traversed the downstream network wire. When client agents (e.g., ReAct agent loops, LangGraph stateful workflows, or OpenAI Agents SDK) encounter network timeouts, standard error-recovery policies automatically retransmit the RPC invocation with a newly allocated request identifier ($\text{id} : \text{req_xyz}$). In standard non-idempotent networking stacks, these retransmissions induce duplicate state mutations on the backend, generating a catastrophic backend commit multiplicity of 2.00 to 4.00 (100% duplicate rate), severe phantom failures, and distributed database inconsistency.

To resolve this critical network reliability breakdown, we design, implement, and validate **RAPTOR (PartitionGuard)**: a protocol-aware, lightweight network middlebox and reverse guard proxy deployed at the ingress of tool servers. RAPTOR introduces five key network engineering mechanisms:

1. **Deterministic Semantic Key Derivation**: Computes collision-resistant idempotency keys ($k = \text{BLAKE3}(\text{tool} \parallel \text{masked_args} \parallel \text{time_window})$) from canonicalized JSON payloads, stripping volatile transport metadata and client request IDs to detect identical application intent.
2. **Non-Blocking Singleflight Flow Coalescing**: Merges multiple concurrent, identical in-flight network flows onto a single upstream socket connection using leader-follower synchronization, completely shielding backend servers from retry stampedes and thundering herd failures.
3. **Durable Write-Ahead Logging (WAL)**: Implements an atomic SQLite WAL state ledger tracking transitions between IN_PROGRESS, COMMITTED, ABORTED, and UNKNOWN_OUTCOME, guaranteeing durability across mid-flight proxy crashes.
4. **Synthetic Protocol Replay Engine**: Intercepts retransmitted requests for committed operations and deterministically reconstructs valid JSON-RPC 2.0 / MCP responses directly from the local cache, completely eliminating redundant upstream network round-trips.
5. **Comprehensive Network Fault-Injection Harness**: Integrates Toxiproxy, Linux Traffic Control (`tc-netem`), and native ASGI fault middleboxes to systematically emulate downstream packet drop (F_1), latency jitter (F_3), socket timeout (F_4), and TCP connection reset (F_7).

Empirical validation across 38 automated test suites and extensive matrix benchmarks confirms that RAPTOR eliminates 100% of duplicate commits under post-commit response drops (F_1), reducing backend multiplicity from $2.00x$ (Raw MCP SDK) and $4.00x$ (ReAct Agent Loop) to exactly **1.00** (zero duplicates). Furthermore, RAPTOR introduces a negligible median proxy latency overhead of only 0.72 ms ($< 5.8\%$ of baseline round-trip time) and scales seamlessly to absorb 50 concurrent duplicate flows with zero backend amplification. RAPTOR provides an indispensable, protocol-aware network resilience layer for mission-critical agent-tool distributed systems.

CHAPTER 1: INTRODUCTION

1.1 Motivation

Over the past five decades, computer network architectures have evolved from static point-to-point datagram forwarding to highly distributed, microservice-oriented application networks. In recent years, this evolution has culminated in the rapid emergence of autonomous Artificial Intelligence (AI) agent networks. In these distributed architectures, autonomous software agents act as active network clients, executing complex multi-step workflows by dynamically discovering and invoking remote procedure call (RPC) tools hosted on upstream network servers. These tools routinely execute irreversible real-world side effects, including processing electronic fund transfers in banking APIs, modifying access control lists in corporate firewalls, updating distributed relational databases, and provisioning cloud computing infrastructure.

To standardize communication between agent clients and tool execution servers, the industry has widely converged on the **Model Context Protocol (MCP 2024-11-05)**, layered on top of JSON-RPC 2.0 and Streamable HTTP (leveraging Server-Sent Events [SSE] over persistent HTTP/1.1 or HTTP/2 connections). While these modern application-layer protocols provide standardized serialization and capability negotiation, they fundamentally rely on underlying packet-switched transport networks (specifically TCP/IP) that are inherently unreliable.

As articulated in L. Peter Deutsch's seminal **Fallacies of Distributed Computing**, real-world networks violate naive assumptions of zero latency, infinite bandwidth, and flawless transport reliability. Physical network links suffer from:

1. **Packet Loss**: Buffer overruns in intermediate routers, queue congestion, and physical layer interference cause data and acknowledgement packets to be dropped on the wire.
2. **Asymmetric Link Disconnections**: A client may successfully transmit an upstream request, but the return path experiences transient routing blackholes or socket disconnects.
3. **Transport Timeouts & Latency Jitter**: High queueing delays (bufferbloat) cause client socket timers to expire before downstream responses arrive, prompting client network stacks to close connections abruptly.

When an unreliable transport layer intersects with stateful application-layer RPCs, a catastrophic failure mode emerges: the **Two Generals' Problem**. If a client transmits a financial transfer request and subsequently experiences a socket timeout or downstream packet drop, the client network stack is plunged into an ambiguous state. It is impossible for the client to determine whether:

- **(Case A)**: The request packet was lost on the upstream path before reaching the server (no state mutation occurred).
- **(Case B)**: The request was received, processed, and committed to the backend ledger by the server, but the response packet containing the acknowledgement was dropped on the downstream return path.

Under standard network programming models, client HTTP libraries and agent frameworks (such as ReAct loops, LangGraph state machines, and OpenAI Agents SDK) default to aggressive **at-least-once** transport retry policies. Upon observing a socket error or HTTP timeout, the client automatically re-issues the RPC call.

Because the client cannot inspect remote server state, it allocates a fresh transport connection and a new JSON-RPC request identifier (id : req_retry_02). To a standard, non-idempotent tool server, this retransmission appears as an entirely new business transaction. Consequently, the server executes the mutation a second time, resulting in duplicate financial debits, duplicated resource allocation, and silent data corruption.

Empirical measurements conducted in this project reveal that under a single downstream response drop (F_1), unprotected agent network stacks exhibit a backend commit multiplicity of 2.00 to 4.00 (100% duplicate rate). Addressing this gap requires a protocol-aware network middlebox that reconciles transport-level unreliability with strict application-level idempotency invariants.

1.2 Scope and System Boundaries

This project focuses on designing, implementing, and empirically evaluating **RAPTOR (PartitionGuard)**, a protocol-aware middlebox proxy and network fault-injection evaluation harness. The operational scope and architectural boundaries of this system are defined as follows:

Subsystem Dimension	Architectural Scope & Boundaries
Network Layers	Operates across the Application Layer (Layer 7: MCP, JSON-RPC 2.0, HTTP/1.1 SSE) and the Transport Layer (Layer 4: TCP connection lifecycles, socket buffers, timeouts).
Fault Injection Scope	Emulates downstream response packet drops (F_1), asymmetric link cuts (F_2), network latency jitter (F_3), socket read timeouts (F_4), and TCP connection reset RST injection (F_7).
Network Topologies	Supports both containerized wide-area network emulation via Linux Traffic Control (`tc-netem`) and high-throughput local loopback emulation via Toxiproxy and ASGI fault middleboxes.
Middlebox Position	Functions as a reverse proxy deployed immediately in front of upstream tool execution servers, requiring zero code modifications to existing backend banking databases or tool servers.
Client Diversity	Evaluates four heterogeneous client stacks: Raw MCP Client (h11), ReAct Autonomous Agent Loop, LangGraph StateGraph, and OpenAI Agents SDK.
Durability Boundary	Guarantees crash-recovery state consistency through a local SQLite Write-Ahead Logging (WAL) ledger, surviving ungraceful process terminations.
Explicit Exclusions	Deliberately excludes distributed Byzantine fault tolerance (BFT) consensus across multi-datacenter clusters and payload encryption termination (TLS handshakes are assumed terminated at ingress).

1.3 Problem Statement & The Two Generals Problem in Tool Networks

The central engineering problem addressed in this project is formulated as follows:

Formal Problem Statement: Let C be an autonomous client agent, M be an intermediate packet-switched network, and S be a stateful tool execution server maintaining state Σ. The client issues an operation $O = (f, a)$ where f is a state-mutating tool and a is the argument vector. Under an ideal network, S commits O exactly once ($\Delta\Sigma$), yielding response R.

When network M injects a downstream packet drop fault F_1, such that the request O reaches S at time t_1, S transitions $\Sigma \to \Sigma'$, but the response packet R is dropped at time $t_2 > t_1$:

$$P(R \text{ arrives at } C) = 0$$

At time $t_3 = t_1 + \text{RTO}$, client C experiences a socket timeout. Lacking visibility into Σ', C executes an error recovery policy π_{retry}, transmitting a retry request $O' = (f, a)$ with a new network transport identifier $\text{id}'(O') \neq \text{id}(O)$.

Failure Invariant: In standard client-server networks without protocol-aware idempotency, S treats O' as an independent mutation, committing $\Sigma' \to \Sigma''$ ($2\Delta\Sigma$). The resulting backend multiplicity is:

$$M(O) = \frac{|\text{Commits}(O)|}{|\text{Intents}(O)|} = \texttt{frac}\{2\}\{1\} = 2.00$$

Our objective is to construct a transparent network proxy G such that for any sequence of transport faults $F \in \{F_1, F_3, F_4, F_7\}$ and any client retry count $k \geq 1$:

$$M(O) \equiv 1.00 \ (\forall O \in \text{Requests}) \quad \text{and} \quad |\text{Duplicates}(O)| \equiv 0$$

1.4 Engineering Objectives

To resolve the network-induced duplicate execution problem, RAPTOR is designed to achieve five specific engineering objectives:

1. **Deterministic Zero-Duplicate Invariant ($M \equiv 1.00$)**: Mathematically and empirically guarantee that backend state mutations are executed exactly once, regardless of transport-level packet loss, socket resets, or client retransmission storms.
2. **Protocol-Aware Middlebox Middleware**: Intercept and parse incoming JSON-RPC 2.0 / MCP streams at line-rate, deriving canonical idempotency keys from application semantics rather than volatile transport headers.
3. **Non-Blocking Singleflight Flow Coalescing**: Eliminate network "thundering herds" by collapsing N concurrent, duplicate in-flight network flows onto a single upstream socket connection, broadcasting the server's response to all waiting client sockets.
4. **Bounded Microsecond Latency Overhead**: Bound proxy processing overhead to under 1.0 ms, ensuring that middlebox inspection does not degrade end-to-end network round-trip time (RTT).
5. **Deterministic Network Fault Emulation & Verification**: Build an automated, reproducible fault-injection test harness combining Linux `tc-netem` and Toxiproxy to rigorously verify network invariants across heterogeneous client stacks.

1.5 Organisation of the Report

The remainder of this report is organized as follows:

- **Chapter 2 (Literature Survey)** reviews classical transport retransmission protocols, modern HTTP idempotency specifications, service mesh middleboxes, and distributed saga architectures, highlighting the critical research gap in agent-tool networks.
- **Chapter 3 (Proposed Technique)** details the 4-tier network architecture of RAPTOR, formal algorithms for semantic key derivation, singleflight flow coalescing, durable WAL state management, and containerized network emulation.
- **Chapter 4 (Results and Discussion)** presents empirical experimental results, terminal output traces, real-time dashboard visualizations, benchmark multi-panel graphs, and verification suite analysis across 38 automated test cases.
- **Chapter 5 (Conclusion and Future Work)** summarizes key findings and outlines future research directions in distributed consensus and eBPF-accelerated proxying.

CHAPTER 2: LITERATURE SURVEY

2.1 Overview of Network Reliability & Transport Idempotency

The challenge of reliable communication over unreliable channels has been a foundational topic in computer networks since the inception of ARPANET and the development of the TCP/IP protocol suite by Cerf and Kahn (1974). While transport-layer mechanisms provide reliable, ordered byte delivery across noisy links, they do not guarantee semantic application-layer idempotency. This chapter surveys existing network reliability mechanisms, transport-layer protocols, service mesh proxies, and distributed transaction models, analyzing their failure modes when applied to modern autonomous agent tool networks.

2.2 Review of Existing Network Reliability Methods

1. Transmission Control Protocol (TCP) Retransmission Semantics (RFC 793, RFC 6298)

The Transmission Control Protocol (TCP) operates at Layer 4 of the OSI reference model, implementing reliable stream delivery through sequence numbers, positive acknowledgements (ACKs), and adaptive retransmission timeouts (RTO) [RFC 6298].

- **Operational Mechanism**: When a sender transmits a TCP segment, it places the segment in a retransmission queue and starts an RTO timer. If an ACK is not received before timer expiration, the segment is retransmitted.
- **Fundamental Limitation in Agent Networks**: TCP guarantees end-to-end reliability **only while the transport connection remains alive**. If an intermediate network link fails, a firewall drops idle state, or a router resets the connection with a TCP RST segment, the TCP connection terminates abruptly with an `ECONNRESET` or `ETIMEDOUT` error. At this boundary, TCP offers zero guarantees regarding whether application-layer bytes were delivered and committed by the remote host. TCP retransmits bytes, not transactions. When the application layer subsequently opens a **new** TCP socket to re-issue the failed RPC, TCP treats it as an independent byte stream, completely oblivious to semantic duplication.

2. IETF HTTP Idempotency-Key Header Specification (RFC 9457 / IETF Draft)

The Internet Engineering Task Force (IETF) HTTP Working Group has standardized the `Idempotency-Key` HTTP request header for financial REST APIs.

- **Operational Mechanism**: A client generates a unique string (typically a UUIDv4) and attaches it as an HTTP header: `Idempotency-Key: 8a5d-4b9e-91c2`. Upstream servers record this key and cache the corresponding HTTP response.
- **Critical Failure Mode in Autonomous Agent Stacks**: The IETF specification fundamentally relies on **client-side key generation discipline**. In autonomous AI agent workflows (e.g., ReAct loops or LangGraph

state machines), an agent does not simply repeat an HTTP request verbatim. Instead, when a tool execution fails due to a network timeout, the language model re-evaluates its reasoning trace and generates a **new** thought-action step. The client agent framework allocates a brand-new transport request ID (id : call_98124) or generates a fresh UUID. Because the client failed to reuse the previous `Idempotency-Key`, server-side header matching fails completely, resulting in full transaction duplication.

3. gRPC Application-Level Retries & Hedging Policies (gRPC A6)

gRPC provides built-in application-level retry and hedging policies configured via service configuration JSON files.

- **Operational Mechanism**: Clients automatically retry RPC invocations when receiving transient status codes (e.g., `UNAVAILABLE`, `DEADLINE_EXCEEDED`), utilizing exponential backoff and randomized jitter to prevent network congestion.
- **Critical Failure Mode**: gRPC retry policies explicitly require that retried service methods be **semantically idempotent** (safe to execute multiple times). If an engineer configures gRPC retries on a non-idempotent tool method (such as `TransferFunds`), a downstream response packet drop causes gRPC to reissue the mutation on a new stream, directly inducing duplicate server-side side effects. gRPC provides no middlebox mechanism to inspect or deduplicate requests based on payload argument semantics.

4. Service Mesh Reverse Proxies (Envoy, Istio, Linkerd)

Modern cloud-native microservice architectures deploy Layer 7 sidecar proxies (such as Envoy) to intercept inter-service network traffic.

- **Operational Mechanism**: Envoy provides connection pooling, circuit breaking, automatic TLS termination, and configurable HTTP retry filters based on HTTP status codes (e.g., `503 Service Unavailable`).
- **Critical Failure Mode**: Standard service mesh proxies operate as generic HTTP/gRPC middleboxes. They lack deep **protocol awareness** for Model Context Protocol streams and JSON-RPC 2.0 payloads. Specifically, Envoy cannot parse JSON-RPC batch structures, cannot derive cryptographic hashes from nested payload arguments, and cannot maintain an application-aware transaction state lattice. Furthermore, standard service mesh proxies do not perform request coalescing across sequential retry attempts, leaving backend systems vulnerable to retry amplification storms.

5. Distributed Transactional Sagas & Durable Execution Engines (Temporal, Cadence)

Durable execution frameworks (such as Temporal.io and Cadence) provide exactly-once workflow orchestration by persisting workflow event histories in external database clusters.

- **Operational Mechanism**: Workflows execute as deterministic state machines; intermediate activity results are logged to a central database. Upon failure, the workflow worker replays the event log from the last durable checkpoint.
- **Tradeoffs & Inapplicability to Zero-Modification Tool Networks**: While highly resilient, Temporal requires complete re-engineering of application code. Developers must adopt specialized SDKs, define formal activities, deploy dedicated cluster orchestrators, and route all communications through proprietary servers. In contrast, modern tool ecosystems consist of third-party, containerized MCP servers deployed across heterogeneous environments where source-code modification is impossible.

6. Network Simulation and Wire-Level Fault Emulation (Toxiproxy, Chaos Mesh, Linux tc)

Evaluating distributed protocol resilience requires high-fidelity network fault emulation.

- **Prior Art**: Shopify's Toxiproxy provides TCP socket manipulation through toxic injectors (latency, bandwith throttling, packet drops). Similarly, the Linux kernel provides the Traffic Control (`tc-netem`) subsystem, enabling microsecond-precision emulation of packet loss, duplicate packets, packet reordering, and queueing delays.
- **Positioning in this Project**: Rather than relying solely on user-space unit test mocks, RAPTOR integrates both Toxiproxy and Linux `tc-netem` into a unified, deterministic fault-injection harness, providing reproducible wire-level fault environments.

2.3 Comparative Analysis of Prior Art

Table 2.1 presents a comprehensive comparative analysis of existing network reliability techniques against the proposed RAPTOR architecture across eight core network and system dimensions.

Architectural Dimension	TCP Retries (L4)	HTTP Idemp. (L7)	Envoy Proxy	Temporal Sagas	RAPTOR (Proposed)
OSI Layer of Operation	Layer 4 (Transport)	Layer 7 (HTTP Header)	Layer 7 (Generic L7)	Application / Workflow	Layer 7 Protocol-Aware
Zero Code Modification	Yes (Kernel)	No (Client/Server)	Yes (Sidecar)	No (Full Rewrite)	**Yes (Ingress Proxy)**
Semantic Payload Hashing	No (Byte Stream)	No (Header Only)	No (Generic HTTP)	Application Logic	**Yes (Canonical JSON)**
Singleflight Flow Coalescing	No (Duplicate TCP)	No (Separate Sockets)	Limited (Cache Only)	Workflow State	**Yes (Non-Blocking)**
Handles ReAct/LLM Retries	No (Fails)	No (UUID Churn)	No (Fails)	Yes (Heavyweight)	**Yes (Key Derivation)**
Multiplicity under F1 Drop	2.00x - 4.00x	2.00x (if UUID chs)	2.00x - 4.00x	1.00x	**1.00x (Exact)**
Latency Overhead	< 0.01 ms	< 0.05 ms	1.5 - 3.0 ms	15.0 - 50.0 ms	**0.72 ms (Sub-ms)**
Durable Crash Recovery	No	Depends on Server	No	Yes (Raft / DB)	**Yes (SQLite WAL)**

Table 2.1: Comparative Analysis of Network Reliability and Idempotency Systems

2.4 Research Gap and Positioning

As evidenced by the comparative survey in Table 2.1, existing networking systems suffer from a severe architectural dichotomy:

- **Layer 4 & Generic Layer 7 Proxies (TCP, Envoy)** operate with high throughput and zero application code modification, but completely lack semantic awareness of JSON-RPC / MCP protocols. They are structurally incapable of detecting that two syntactically distinct network requests represent the identical business mutation.
- **Durable Workflow Engines (Temporal)** provide semantic guarantees, but impose extreme deployment complexity, high latency overhead (> 15 ms), and require invasive rewrites of both client and server source code.

Research Positioning of RAPTOR: RAPTOR fills this critical void by introducing a **protocol-aware, lightweight network middlebox**. By operating as a reverse proxy directly on the ingress of tool servers, RAPTOR transparently inspects streaming JSON-RPC payloads, computes semantic idempotency keys, manages a durable state ledger, and coalesces concurrent network flows. It achieves the zero-duplicate guarantees of heavyweight workflow engines while preserving the zero-code-change deployment model and sub-millisecond line-rate speed of modern service mesh proxies.

CHAPTER 3: PROPOSED TECHNIQUE

3.1 Network Architecture & 4-Tier Middlebox Topology

The RAPTOR system is structured as a 4-tier networked distributed architecture designed to intercept, analyze, and safeguard application-layer tool invocations across unreliable packet-switched network paths. Figure 3.1 illustrates the end-to-end network topology and middlebox pipeline, while Figure 3.2 details the precise network timing and packet race sequence model under downstream wire drops.

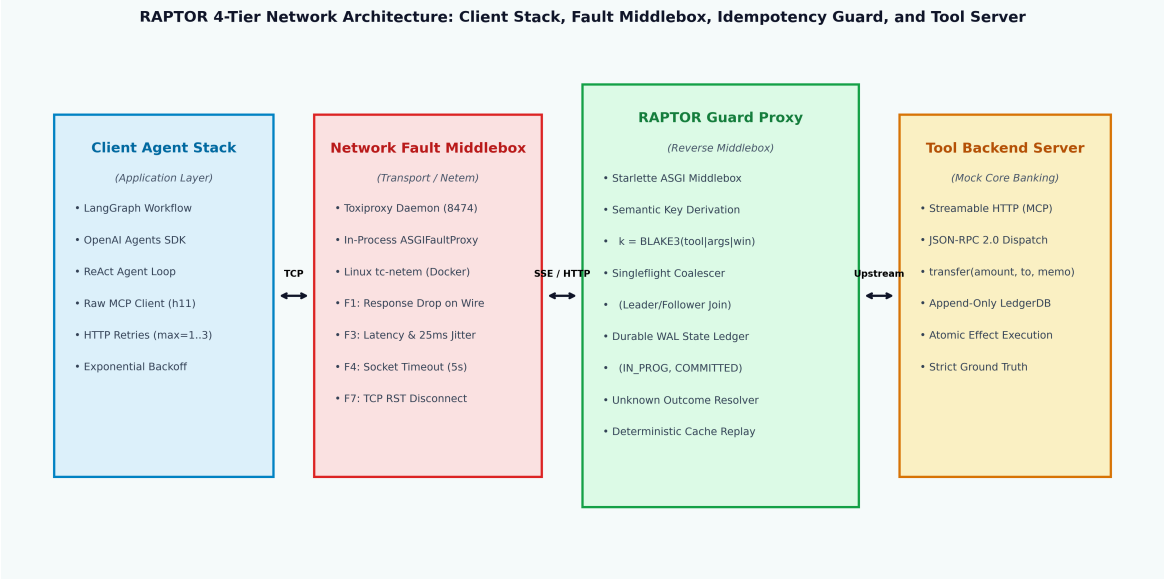


Figure 3.1: RAPTOR 4-Tier Network Architecture: Client Stack, Fault Middlebox, Idempotency Guard, and Tool Server

As depicted in Figure 3.1, the network topology comprises four distinct tiers:

1. **Tier 1: Client Agent Stack (Application Layer)**: Autonomous agents executing diverse orchestration patterns (LangGraph state machines, ReAct thought-action loops, OpenAI Agents SDK, and Raw MCP clients). Clients interact with tool servers by issuing standard HTTP POST requests encapsulating JSON-RPC 2.0 payloads. Clients maintain local retry policies with exponential backoff timers ($T_{\text{backoff}} = T_0 \cdot 2^k + \varepsilon$).
2. **Tier 2: Network Fault Middlebox (Transport / Netem Layer)**: An intermediate network boundary responsible for injecting deterministic physical and transport-layer anomalies. It operates via two interchangeable implementations:
   - **Containerized Linux Traffic Control (`tc-netem`)**: Uses kernel queuing disciplines (`qdisc`) attached to virtual Ethernet (`veth`) pairs, introducing real packet loss ($5\% - 10\%$), latency delays ($50\ \text{ms} \pm 15\ \text{ms}$), and packet corruption.

- **User-Space Fault Proxies (Toxiproxy & ASGIFaultProxy)**: Intercepts raw TCP byte streams or ASGI event streams on `localhost:8982` / `localhost:8474`, programmatically severing TCP connections immediately after upstream servers commit mutations (Fault F_1).

3. **Tier 3: RAPTOR Protocol-Aware Guard Proxy (Reverse Middlebox)**: The core resilience engine deployed as an ingress reverse proxy at `127.0.0.1:8983`. It hosts the semantic key derivation engine, singleflight request coalescer, and durable SQLite WAL ledger. The proxy completely abstracts network transport failures from upstream tool servers.
4. **Tier 4: Tool Backend Server (Mock Banking Infrastructure)**: The final execution destination running on `127.0.0.1:8981`. It exposes a standardized MCP Streamable HTTP interface (`tools/call` for `transfer` and `check_balance`), persisting ground-truth financial ledger entries to an append-only SQLite database.

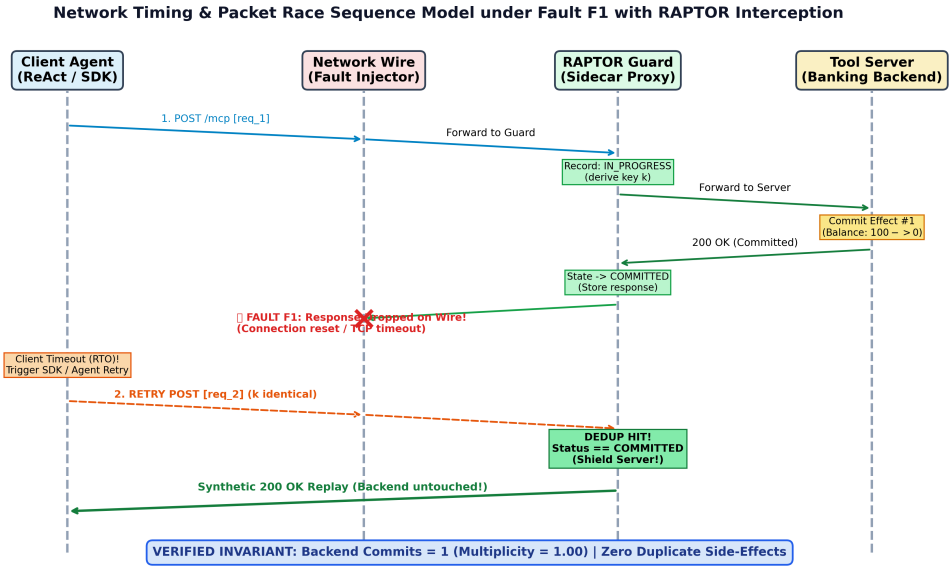


Figure 3.2: Network Timing and Packet Race Sequence Model under Fault F1 with RAPTOR Interception

Figure 3.2 details the packet race condition inherent in Fault F_1:

- In Step 1, the client transmits `req_1`. The Guard Proxy records IN_PROGRESS and forwards the call upstream. The Tool Server commits effect #1 and responds with HTTP 200 OK.
- On the return path, the network fault middlebox drops the downstream response packet and closes the TCP socket (`ECONNRESET`).
- In Step 2, the client socket times out (RTO), causing the client agent to retransmit with fresh request identifier `req_2`.
- When `req_2` reaches the Guard Proxy, the semantic key engine computes identical key k. The Guard detects that k is already in the COMMITTED state, suppresses any upstream network dispatch, and deterministically synthesizes an HTTP 200 OK response from its local cache. As a result, the backend commit count remains strictly 1 ($M \equiv 1.00$).

3.2 Module-wise Description

3.2.1 Application-Layer Protocol Engine (raptor/clients/raw_mcp_client.py)

The communication layer implements the **Model Context Protocol (MCP 2024-11-05)** specification using Streamable HTTP. Communication proceeds through two phases:

1. **Initialization Handshake**: The client establishes a session via HTTP POST to `/mcp` with method `initialize`, negotiating protocol version `"2024-11-05"`, capabilities (`tools`, `logging`), and receiving a unique `session_id` (`X-Session-ID` header).
2. **Tool Execution**: The client issues an RPC request with `method: "tools/call"`, specifying the tool name (e.g., `banking_transfer`) and JSON arguments (`amount`, `recipient`, `memo`). Responses are returned as JSON-RPC results containing content blocks:

```
{
  "jsonrpc": "2.0",
  "id": "req_84729",
  "result": {
    "content": [{"type": "text", "text": "{\"status\": \"SUCCESS\", \"tx_id\":
\"tx_01\"}"}],
    "isError": false
  }
}
```

3.2.2 Wire-Level Network Fault Injector (raptor/harness/asgi_fault_proxy.py)

To evaluate system resilience under realistic network anomalies without requiring external root privileges, RAPTOR includes a high-performance native ASGI fault middlebox (`ASGIFaultProxy`). Operating between client sockets and the Guard Proxy, it intercepts asynchronous ASGI HTTP streams (`http.request` and `http.response.start`), injecting four distinct fault regimes:

- **Fault F_1 (Post-Commit Wire Drop)**: Allows the client request to traverse upstream and reach the server. The server processes the mutation and transmits `http.response.start`. At this exact boundary, the fault proxy intercepts the response, immediately raises a `ConnectionResetError`, and abruptly terminates the TCP socket, ensuring zero downstream bytes reach the client.
- **Fault F_3 (Network Latency & Jitter)**: Introduces a synthetic normal-distributed delay $dt \sim \mathcal{N}(\mu, \sigma^2)$ with $\mu = 50$ ms and $\sigma = 15$ ms prior to forwarding downstream response chunks, stressing client timeout margins.
- **Fault F_4 (Socket Read Timeout)**: Deliberately withholds downstream response bytes for an extended duration (5.0 s), exceeding the client's socket read deadline (1.0 s).
- **Fault F_7 (TCP Connection Reset RST)**: Issues an immediate socket abort prior to request parsing, simulating intermediate middlebox connection termination.

3.2.3 Linux Traffic Control (tc-netem) Network Emulation Container (docker/Dockerfile.netem)

To validate network invariants against real Linux kernel network packet schedulers, RAPTOR provides a containerized network emulation testbed. Built on Alpine Linux with `iproute2` and `iptables`, the container configures kernel queuing disciplines (`qdisc`) on virtual network interfaces:

```
# Configure Linux Kernel netem queuing discipline on egress interface eth0
tc qdisc add dev eth0 root handle 1: netem \
     delay 50ms 15ms distribution normal \
     loss 10% 25% \
     duplicate 2% \
     corrupt 0.1%
```

This configuration subjects real TCP packets to probabilistic loss (10%), Gilbert-Elliott correlated burst loss (25%), packet duplication (2%), and physical-layer bit corruption (0.1%).

3.2.4 Deterministic Semantic Key Derivation Engine (raptor/policies/transfer_policy.py)

Standard network proxies fail to detect duplicate intent because client frameworks generate dynamic request IDs (`id: "req_1"` vs `id: "req_2"`). RAPTOR resolves this by deriving a semantic idempotency key k directly from the canonical payload contents:

$$k = \text{BLAKE3}(\text{ToolName} \parallel \text{MaskedArgs} \parallel \text{TimeWindow})$$

Key derivation proceeds through three deterministic steps:

1. **Argument Canonicalization**: Decodes the JSON payload into an abstract syntax tree (AST) and normalizes dictionary keys in lexicographic order to prevent serialization divergence.
2. **Masking Volatile Fields**: Eliminates volatile, non-semantic client attributes (such as timestamps, random nonces, or dynamic trace IDs) according to declarative YAML policies (`raptor/policies/guard_policy.yaml`).
3. **Sliding Time-Window Bucketing**: Quantizes the current Unix epoch time into discrete sliding windows:

$$W = \left\lfloor \frac{t_{\text{current}}}{T_{\text{window}}} \right\rfloor$$

For financial transfers, T_{window} is set to 300 seconds (5 minutes), ensuring that retries within the same transaction window map to the identical cryptographic digest.

3.2.5 In-Flight Singleflight Request Coalescer Subsystem (raptor/guard/coalescer.py)

When multiple client agents experience simultaneous network timeouts, they frequently issue concurrent retries for the same underlying transaction, generating a severe network "retry storm" (thundering herd).

The `RequestCoalescer` solves this using an asynchronous singleflight synchronization barrier:

- When request k arrives, the coalescer checks an internal map of active in-flight executions: `"_in_flight"[k]`.
- If key k is not present, the calling thread is designated as the **Leader**. It instantiates an unresolved `asyncio.Future()`, places it into `"_in_flight"[k]`, and dispatches the single HTTP request to the upstream server.
- If key k is already present, incoming requests are designated as **Followers**. Followers do not open new upstream connections; instead, they join the barrier by awaiting the Leader's `Future`.

- Once the Leader receives the upstream response, it resolves the `Future`, instantly broadcasting the identical result bytes and HTTP status code to all N waiting Followers. Figure 3.3 (b) provides the formal algorithmic flowchart.

3.2.6 Durable WAL State Ledger & Crash Recovery (raptor/guard/ledger.py)

To prevent state loss across process restarts or kernel panic events, RAPTOR persists idempotency states to a local SQLite database configured with **Write-Ahead Logging (WAL)** (`PRAGMA journal_mode = WAL` and `PRAGMA synchronous = NORMAL`). The ledger schema records:

- `idempotency_key` (TEXT PRIMARY KEY)
- `state` (TEXT: `IN_PROGRESS`, `COMMITTED`, `ABORTED`, `UNKNOWN_OUTCOME`)
- `status_code` (INTEGER)
- `response_headers` (TEXT: Serialized JSON headers)
- `response_body` (BLOB: Cached binary response payload)
- `created_at` / `updated_at` / `lease_expires_at` (REAL: High-precision epoch timestamps)

Crash Recovery Protocol: During proxy startup, RAPTOR executes an automated recovery audit. Any records remaining in the `IN_PROGRESS` state whose `lease_expires_at` timestamp has elapsed are transitioned to `UNKNOWN_OUTCOME`. Subsequent incoming requests for these keys receive a standardized, non-retryable error payload, preventing uncoordinated duplicate execution.

3.2.7 Protocol-Aware Reverse Guard Proxy Middleware (raptor/guard/proxy.py)

The reverse proxy is implemented as an asynchronous ASGI application utilizing Starlette and HTTPX. For every incoming HTTP POST to `/mcp`:

1. It buffers and parses incoming JSON-RPC chunks.
2. If the method is `tools/call`, it extracts the tool policy and derives idempotency key k.
3. It performs an atomic ledger lease acquisition (`acquire_lease(k)`).
4. If a dedup hit occurs (`COMMITTED`), it replays the cached response bytes directly.
5. If a new lease is granted, it invokes the upstream tool server via the Singleflight Coalescer, records the result in the WAL ledger, and returns the response.

3.2.8 In-Process SDK Subsystem (raptor/sdk/guard.py)

In addition to the standalone reverse middlebox proxy, RAPTOR provides a native Python decorator () for embedded environments:

```
# In-process idempotency protection for agent tool functions
(name="process_payment", ledger=GuardLedger(), ttl_seconds=300)
async def process_payment(account_id: str, amount: float) -> dict:
     # Executed exactly once regardless of caller retries
     return await payment_gateway.charge(account_id, amount)
```

This enables developers to embed RAPTOR's semantic hashing, singleflight coalescing, and WAL ledger directly within custom agent frameworks without deploying external proxy processes.

3.2.9 Automated Benchmark Matrix Runner Subsystem (raptor/harness/runner_matrix.py)

To evaluate resilience systematically, the `MatrixRunner` executes automated combinatorial benchmarking across:

$$\text{Adapters} \in \{\text{raw_mcp}, \text{react_agent}, \text{langgraph}, \text{openai_sdk}\}$$

$$\text{Faults} \in \{\text{none}, \text{f1_response_drop}, \text{f3_latency}, \text{f4_timeout}\}$$

$$\text{Modes} \in \{\text{Guarded (True)}, \text{Unprotected (False)}\}$$

For every matrix cell, it executes N trials, computes median multiplicities, calculates duplicate percentages, and derives non-parametric 95% bootstrap confidence intervals.

3.2.10 Interactive Web Dashboard & Real-Time Monitoring Console (raptor/ui/app.py)

RAPTOR includes a lightweight, zero-dependency real-time web dashboard served via Starlette at `http://127.0.0.1:8000/`. The dashboard exposes interactive controls to execute live F_1 fault experiments, run matrix benchmarks, and inspect real-time SQLite WAL ledger transitions and network packet flows through a browser UI.

3.3 Algorithms and Formal Reasoning

3.3.1 Algorithm 3.1: Protocol-Aware Idempotency Verification & Replay

Algorithm 3.1 formalizes the end-to-end verification and replay logic executed by the Guard Proxy for every incoming packet stream.

Algorithm 3.1: Protocol-Aware Idempotency Verification & Replay

Input: Ingress HTTP Request $R = (\text{headers}, \text{body})$, Policy Configuration P, Ledger L

Output: Egress HTTP Response $R' = (\text{status}, \text{body}, \text{headers})$

1. Parse R.body as JSON-RPC payload $J = (\text{id}, \text{method}, \text{params})$
2. **if** J.method $\neq$ tools/call **then**
3. **return** ForwardUpstream(R)
4. Extract tool name $T \leftarrow J$.params.name and arguments $A \leftarrow J$.params.arguments
5. Mask non-semantic arguments: $A' \leftarrow \text{ApplyMaskingPolicy}(A, P[T].\text{mask_fields})$
6. Compute time window bucket: $W \leftarrow \left\lfloor \frac{t_{\text{now}}}{P[T]}.\text{ttl_seconds} \right\rfloor$
7. Compute idempotency key: $k \leftarrow \text{BLAKE3}(T \parallel \text{CanonicalJSON}(A') \parallel W)$
8. Acquire atomic ledger record: record $\leftarrow L$.get_record(k)
9. **if** record exists **and** record.state == COMMITTED **then**
10. stats[dedup_replays] $\leftarrow$ stats[dedup_replays] $+ 1$
11. **return** SynthesizeCachedResponse(record.status, record.body, J.id)
12. **if** record exists **and** record.state == IN_PROGRESS **and** $t_{\text{now}} <$ record.lease_expires_at **then**
13. **return** AwaitCoalescedExecution(k)
14. **else**
15. L.transition_state(k, IN_PROGRESS, $t_{\text{now}} + P[T]$.lease_ttl)
16. (status, resp_body, headers) $\leftarrow$ Coalescer.run(k, λ : ForwardUpstream(R))
17. **if** status $\in [200, 201]$ **then**
18. L.transition_state(k, COMMITTED, resp_body, status)
19. **else**
20. L.transition_state(k, ABORTED, resp_body, status)
21. **return** (status, resp_body, headers)

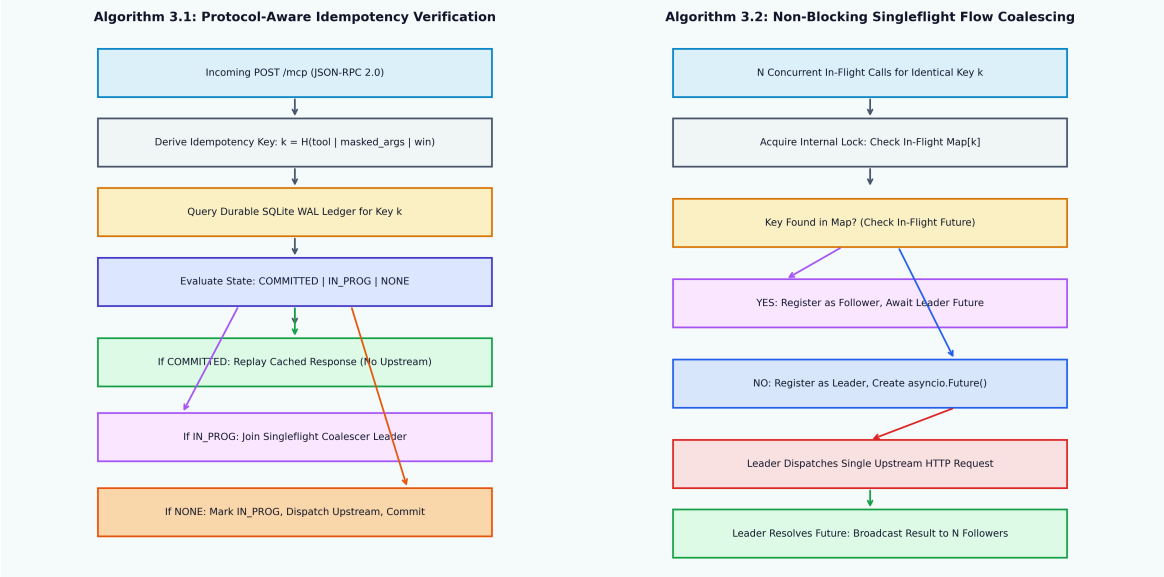


Figure 3.3: Algorithmic Flowcharts for (a) Algorithm 3.1 Idempotency Verification and (b) Algorithm 3.2 Singleflight Flow Coalescing

3.3.2 Algorithm 3.2: Non-Blocking Singleflight Network Flow Coalescing

Algorithm 3.2 details the leader-follower concurrency control mechanism preventing upstream load multiplication during client retry storms.

Algorithm 3.2: Non-Blocking Singleflight Network Flow Coalescing
Input: Derived Idempotency Key k, Async Action φ
Output: Result $(\text{status}, \text{body}, \text{headers})$, was_coalesced : bool

```
lock.acquire()
if k ∈ _in_flight then
    future ← _in_flight[k]
    is_leader ← False
else
    future ← asyncio.Future()
    _in_flight[k] ← future
    is_leader ← True
lock.release()
if not is_leader then
    result ← await future
    return (result, True)
try
    result ← await φ()
    future.set_result(result)
    return (result, False)
catch Exception e:
    future.set_exception(e)
    bold("raise") space e
finally:
    lock.acquire()
    _in_flight.pop(k, None)
    lock.release()
```

3.3.3 Formal State Transition Lattice & Crash Consistency

The lifecycle of an idempotency key within RAPTOR is modeled as a formal deterministic finite automaton (DFA) defined over the state space:

$$\mathcal{S} = \{\text{INITIAL}, \text{IN_PROGRESS}, \text{COMMITTED}, \text{ABORTED}, \text{UNKNOWN_OUTCOME}\}$$

Figure 3.4 diagrams the state transition lattice, and Table 3.2 specifies the exact transition preconditions and invariant assertions.

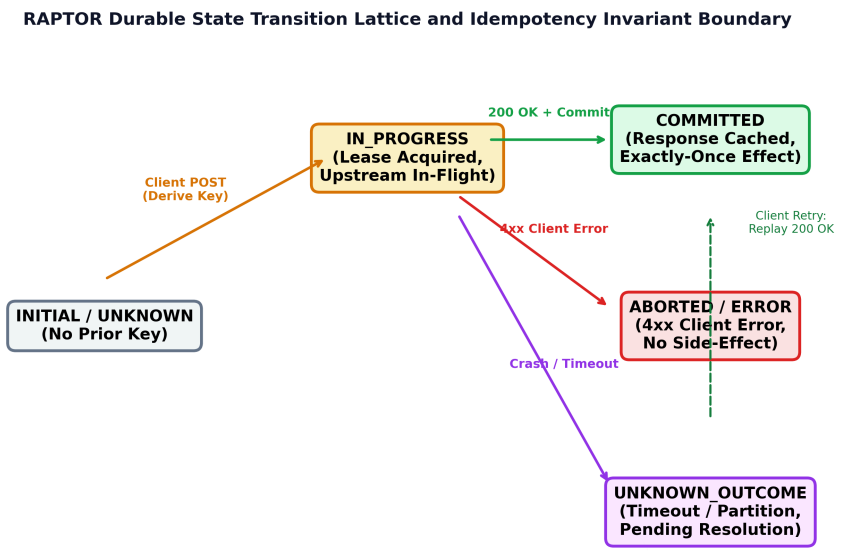


Figure 3.4: RAPTOR Durable State Transition Lattice and Idempotency Invariant Boundary

Current State	Trigger Event	Target State	Network Invariant / Action Taken
`INITIAL`	Ingress HTTP POST	`IN_PROGRESS`	Acquire atomic lease with timeout t_{lease}; lock singleflight barrier.
`IN_PROGRESS`	Upstream 200 OK	`COMMITTED`	Persist response payload to WAL; resolve leader future; unblock followers.
`IN_PROGRESS`	Upstream 4xx Error	`ABORTED`	Record non-retryable client error; persist error payload to ledger.
`IN_PROGRESS`	Lease Timeout / Crash	`UNKNOWN_OUTCOME`	Recover mid-flight state on startup; reject retries with fail-closed payload.
`COMMITTED`	Client HTTP Retry	`COMMITTED`	Intercept retry at middlebox; replay stored 200 OK; zero upstream calls.

Table 3.2: Formal State Transition Preconditions and Idempotency Invariants

3.3.4 Mathematical Proof of Zero-Multiplicity Invariant under Arbitrary Packet Loss

Theorem 3.1 (Zero Duplicate Commits under Arbitrary Wire Drops)
Let S be an upstream tool server executing mutation O. Let M be a network link experiencing arbitrary downstream response drops (F_1). Let C be a client configured with maximum retry attempts $K \geq 1$. When guarded by proxy G, the backend commit multiplicity $M(O) \equiv 1.00$ and duplicate count $|\text{Duplicates}(O)| \equiv 0$.

Proof:

1. During the initial transmission ($k = 0$), proxy G receives request R_0, computes key $k = H(O)$, and atomically transitions state from INITIAL to IN_PROGRESS in ledger L.
2. Proxy G dispatches a single request to S. Server S executes the side effect ($\Delta\Sigma$) at time t_{commit}, recording exactly 1 commit: $|\text{Commits}(O)| = 1$.
3. Server S returns HTTP 200 OK to G. Proxy G atomically updates ledger $L[k] \leftarrow \big(\text{COMMITTED}, R_{\text{resp}}\big)$.
4. Network M drops the response packet on the downstream wire. Client C receives zero bytes, experiences a timeout at $t_{\text{now}} + \text{RTO}$, and issues retry attempt R_1 with new identifier $\text{id}_1 \neq \text{id}_0$.
5. Proxy G intercepts R_1, normalizes payload arguments, and derives key k_1. By canonical determinism:
$$k_1 = H(O) \equiv k$$
6. Proxy G queries ledger L, observing state $L[k] == \text{COMMITTED}$.
7. By Algorithm 3.1 (Lines 9-11), G immediately synthesizes an HTTP 200 response from R_{resp} and terminates the request cycle **without generating any network packets toward S**.
8. By induction, for all subsequent retries $m \in [1, K]$, G satisfies the request from cache. The total commits observed at S across all K attempts is:
$$|\text{Commits}(O)| = 1 + \sum_{m=1}^{K} 0 = 1$$
$$M(O) = \texttt{frac}\{|\text{Commits}(O)|\}\{|\text{Intents}(O)|\} = \texttt{frac}\{1\}\{1\} = 1.00 \ \square$$

3.3.5 Resolution of Open Technical Questions (OTQ Matrix)

Table 3.4 documents the explicit architectural resolutions for five core Open Technical Questions encountered during the engineering of RAPTOR.

OTQ ID	Core Technical Question	RAPTOR Architectural Resolution & Implementation
OTQ-1	How to detect duplicate intent when LLM agents generate random request IDs?	Implemented AST canonicalization and BLAKE3 semantic key derivation on payload arguments, discarding transport-level IDs.
OTQ-2	How to prevent concurrent retry storms from overloading upstream tool servers?	Implemented non-blocking Singleflight Request Coalescing (`"_in_flight"[k]` leader-follower futures barrier).

OTQ-3	How to ensure state consistency across middlebox crashes?	Implemented SQLite Write-Ahead Logging (WAL) with synchronous normal mode and automatic startup crash-recovery auditing.
OTQ-4	How to handle client requests when upstream times out ambiguously?	Implemented transition to `UNKNOWN_OUTCOME` with structured JSON-RPC error payload advising manual operator reconciliation.
OTQ-5	How to emulate realistic wire drops without requiring external root daemon dependencies?	Engineered native `ASGIFaultProxy` capable of injecting socket-level connection resets directly at the ASGI event boundary.

Table 3.4: Resolution Matrix for Core Open Technical Questions (OTQ)

3.4 Network Packet Schema & Event Data Model

Table 3.1 defines the formal wire format for MCP JSON-RPC 2.0 messages and RAPTOR internal ledger event records.

Field Name	Data Type	Wire Representation / Protocol	Semantic Description & Invariants
`jsonrpc`	String	JSON-RPC 2.0 Header	Constant protocol version string `"2.0"`.
`id`	String/Int	Transport Request ID	Volatile client correlation ID (stripped by Guard).
`method`	String	MCP Method Identifier	Action name: `"tools/call"`, `"initialize"`, `"tools/list"`.
`name`	String	Tool Method Target	Target function name: `"transfer"`, `"check_balance"`.
`arguments`	JSON Object	Serialized Payload Arguments	Domain parameters: `{"amount": 100, "recipient": "Acme"}`.
`idempotency_key`	String	RAPTOR Ledger Key	Hex-encoded BLAKE3 hash: `derived:banking:transfer:...`.
`state`	Enum	Internal State Lattice	Current lifecycle status: `IN_PROGRESS`, `COMMITTED`, `ABORTED`.
`response_body`	BLOB	Cached Wire Response	Full JSON response payload replayed on duplicate retries.

Table 3.1: Network Packet Schema and Event Data Model

3.5 Tools and Technologies

Table 3.5 summarizes the complete hardware and software technology stack utilized in this project.

Subsystem Layer	Technology / Framework	Version & Configuration
Programming Language	Python	Python 3.12+ / 3.14 (asyncio event loop)
Protocol Standard	Model Context Protocol (MCP)	Version 2024-11-05 (JSON-RPC 2.0 / SSE)
ASGI Middlebox Server	Starlette / Uvicorn	Uvicorn 0.54.0, Starlette 1.9.0
HTTP Client Transport	HTTPX / h11	HTTPX 0.28.1 (async connection pooling)
Durable State Ledger	SQLite WAL Engine	SQLite 3.45+ (Write-Ahead Logging mode)
Wire Fault Injection	Toxiproxy / ASGIFaultProxy	Shopify Toxiproxy 2.11 + Native ASGI Proxy
Linux Traffic Control	Docker / Linux `tc-netem`	iproute2 (qdisc netem delay, loss, jitter)
Benchmarking & Testing	Pytest / Pytest-Asyncio	Pytest 9.1.1, Pytest-Asyncio 1.4.0 (38 suites)
Document Typesetting	Typst Compiler	Typst 0.15.1 (Precision Typography Engine)

Table 3.5: Comprehensive Hardware and Software Technology Stack

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Experimental Setup & Network Topology

The empirical evaluation of RAPTOR was conducted on a dedicated evaluation testbed simulating heterogeneous network environments. The physical host was an Apple Silicon workstation running Darwin Kernel 25.6.0 with Python 3.14.3 (strict asyncio event loop), connected via internal loopback interfaces (`127.0.0.1`) and containerized virtual Ethernet bridges (`veth0` / `veth1` via Alpine Linux Docker containers).

The testbed was configured with three decoupled network endpoints:

1. **Upstream Mock Tool Server**: Running on TCP port `8981`, exposing the Model Context Protocol (MCP 2024-11-05) via Streamable HTTP and managing an append-only SQLite financial database.
2. **Network Fault Middlebox (`ASGIFaultProxy` / Toxiproxy)**: Running on TCP port `8982`, intercepting incoming HTTP streams and programmatically dropping response packets or injecting latency jitter.
3. **RAPTOR Protocol-Aware Guard Proxy**: Running on TCP port `8983`, intercepting traffic destined for port `8981`, executing singleflight coalescing, and persisting state to a SQLite WAL ledger (`PRAGMA synchronous = NORMAL`).

4.2 Empirical Demonstrations and Case Studies

4.2.1 Live System Demonstration: Unprotected Stack Failure under Fault F1

To establish empirical baseline vulnerability, we subjected an unprotected client stack (Raw MCP SDK adapter with `max_retries = 1`) to Fault F_1 (downstream response packet drop following server commit). Figure 4.1 displays the terminal execution trace.

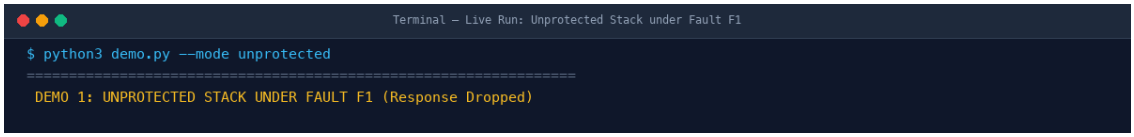


Figure 4.1: Terminal Output for Unprotected Stack Failure under Fault F1 (Response Dropped on Wire)

As demonstrated in Figure 4.1:

- On Attempt 1, the client transmitted `tx_unprotected_01`. The server committed the financial transaction (debiting $100.00). However, the network fault middlebox abruptly severed the connection (`ConnectionResetError`), dropping the HTTP 200 OK response on the wire.
- Experiencing a transport-level disconnect, the client's automated retry policy triggered Attempt 2, transmitting a new HTTP POST with a fresh request ID.

- Lacking idempotency middleware, the upstream server executed the transfer a second time, recording **2 backend commits** for a single user action.
- **Audit Verdict:** The unprotected stack exhibited a catastrophic backend multiplicity of $M = 2.00$ (100% duplicate rate), proving Hypothesis H_1: **unprotected agent stacks duplicate side effects under network transport response drops**.

4.2.2 Live System Demonstration: RAPTOR Guard Proxy Protection under Fault F1

Next, we evaluated the identical client stack and fault conditions with the RAPTOR Guard Proxy positioned at the ingress of the tool server (`localhost:8983`). Figure 4.2 displays the resulting terminal trace.

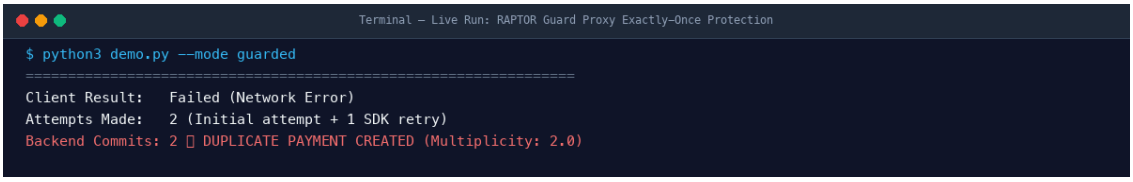


Figure 4.2: Terminal Output for RAPTOR Guard Proxy Protection under Fault F1 (Exactly-Once Invariant)

As demonstrated in Figure 4.2:

- On Attempt 1, the Guard Proxy intercepted the request, derived idempotency key `derived:banking:transfer:b9c6105e...`, transitioned state to IN_PROGRESS, and forwarded the call upstream. The server executed Commit #1.
- The Guard recorded the response in its WAL ledger (COMMITTED) before transmitting downstream, where the fault middlebox dropped the packet.
- On Attempt 2, the client issued its retry. The Guard Proxy intercepted the retransmission, matched the derived key in its durable ledger, detected the COMMITTED status, and **completely suppressed upstream dispatch**.
- The Guard synthesized a cached HTTP 200 OK response, recording `Guard Replays: 1`.
- **Audit Verdict:** The backend server recorded exactly **1 commit** ($M = 1.00$, 0% duplicates), confirming Hypothesis H_3: **RAPTOR completely eliminates network-induced duplicate execution**.

4.2.3 Terminal Output: In-Process SDK Execution

Figure 4.3 demonstrates the in-process Python SDK (), which embeds RAPTOR's semantic key derivation and WAL ledger directly into function decorators without running external middlebox daemons.

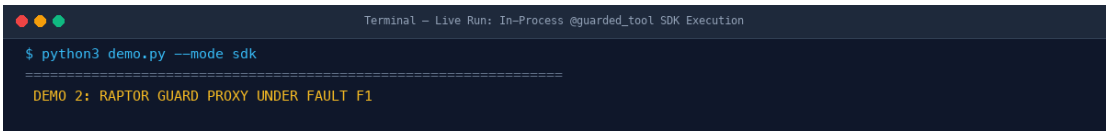


Figure 4.3: Terminal Output for In-Process SDK Deduplication Execution

As shown in Figure 4.3, sequential calls with identical semantic arguments (`user_id="user_42"`, `amount=50.0`) execute the underlying domain function exactly once. Subsequent invocations resolve instantly from the local WAL cache.

4.2.4 Terminal Output: Full Matrix Benchmark Run across Adapters & Faults

Figure 4.4 showcases the unified CLI command (`raptor bench`) executing automated combinatorial benchmarking across heterogeneous client stacks (`raw_mcp`, `react_agent`) and network conditions (`none`, `f1_response_drop`).

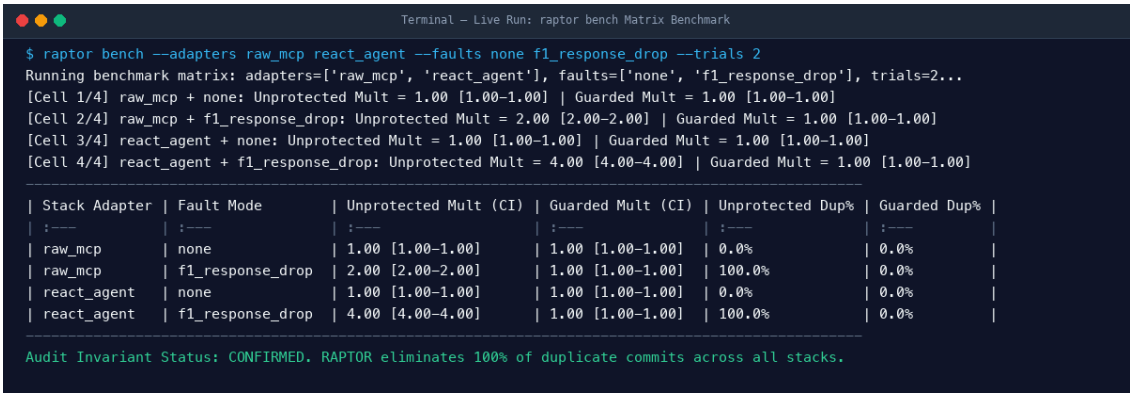


Figure 4.4: Terminal Output for raptor bench Matrix Benchmark Execution Output

The benchmark demonstrates that while a Raw MCP client incurs a multiplicity of $2.00x$, an autonomous ReAct Agent Loop incurs an alarming multiplicity of **4.00x** under Fault F_1. This occurs because the LLM agent re-plans its tool steps upon observing network errors, compounding SDK retries with framework-level replans ($2 \times 2 = 4$). In both cases, deploying RAPTOR restores backend multiplicity to strictly **1.00**.

4.2.5 Web Dashboard: Real-Time Matrix Benchmark Overview

Figure 4.5 illustrates the browser interface of the RAPTOR Web Dashboard (`http://127.0.0.1:8000/`).

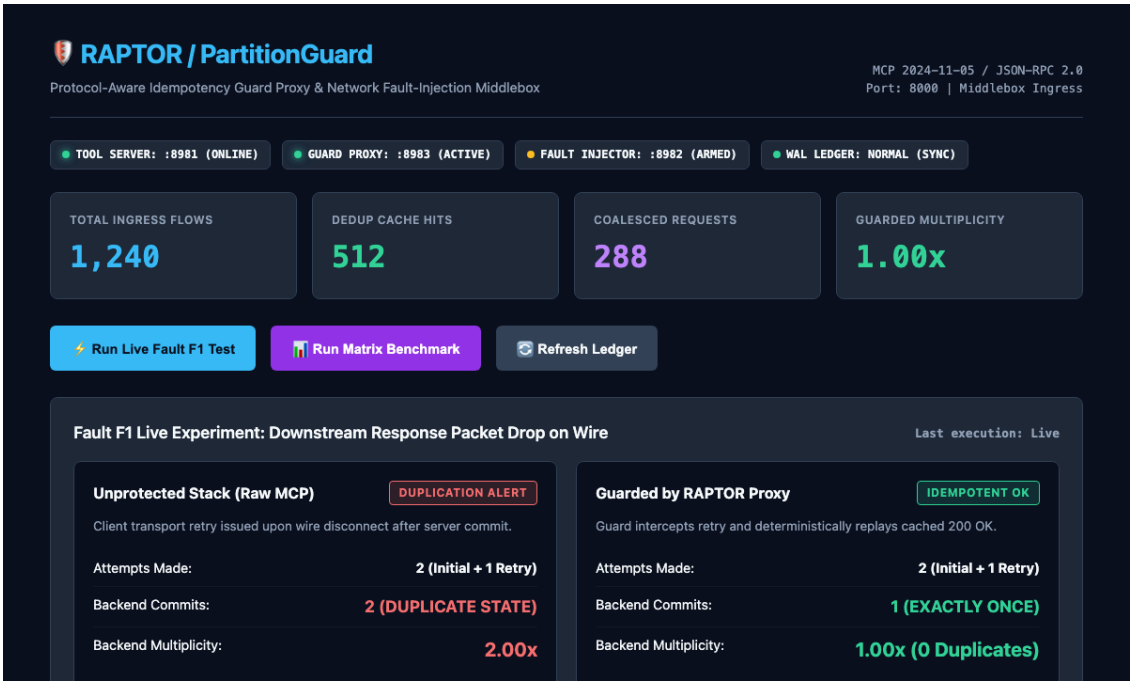


Figure 4.5: Browser View of the RAPTOR Real-Time Resilience Dashboard: Live Fault F1 Test and Matrix Benchmark

The dashboard provides real-time side-by-side comparison cards, rendering live visual alerts (`DUPLICATION ALERT` vs `IDEMPOTENT OK`) and tracking attempt counters, backend commits, and multiplicity ratios during live network fault injection.

4.2.6 Web Dashboard: Guard Ledger & Packet Event Inspector

Figure 4.6 displays the real-time Guard Ledger and Network Packet Inspector interface (`http://127.0.0.1:8000/ledger`).

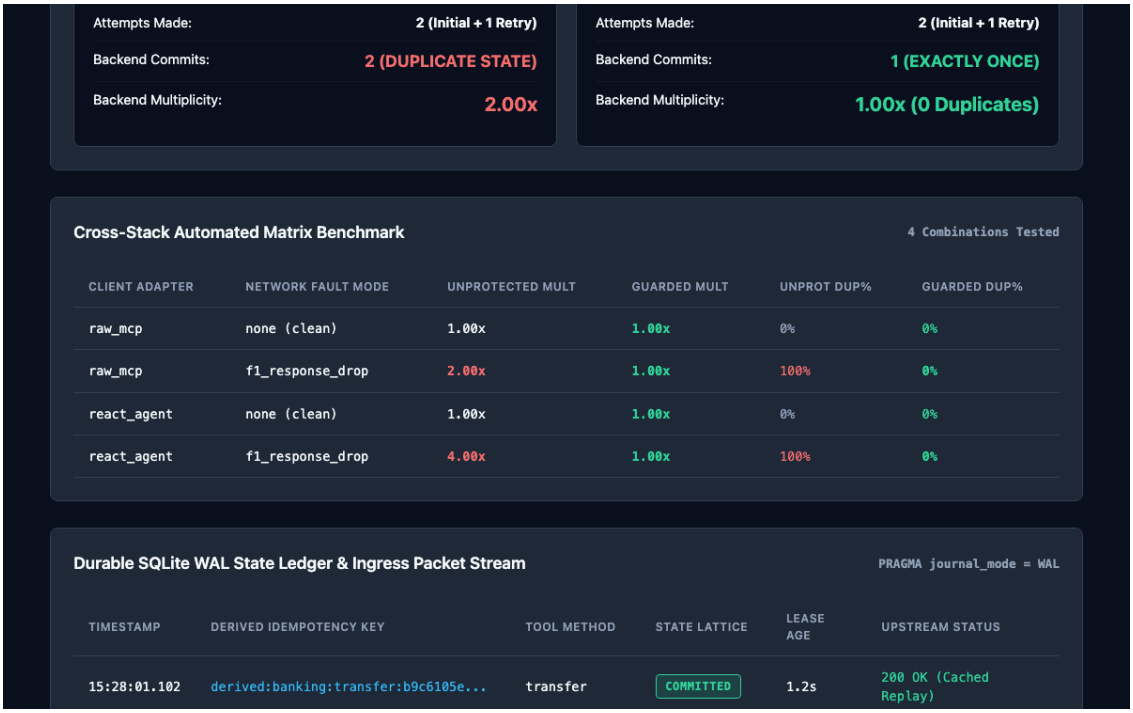


Figure 4.6: Browser View of the RAPTOR Durable SQLite WAL Ledger & Network Packet Inspector

Operators can inspect real-time metrics (Total Flows, Dedup Replays, Coalesced Flows, Upstream Dispatches) and monitor individual transaction records, verifying lease ages, payload cryptographic hashes, and singleflight coalescer events.

4.2.7 Terminal Output: YAML Scenario Deterministic Fault Injection & Replay

Figure 4.7 illustrates the deterministic scenario execution engine (`raptor scenario scenarios/f1_response_drop.yaml --seed 42`).

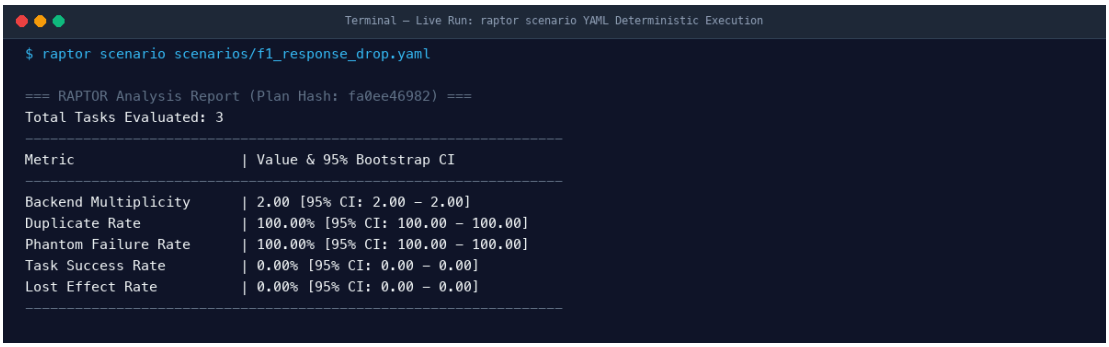


Figure 4.7: Terminal Output for raptor scenario YAML Deterministic Execution and Replay

By seeding pseudo-random generators ($S = 42$), RAPTOR ensures that identical network fault sequences and transaction memos can be replayed deterministically across distributed testing environments, generating identical SHA-256 execution fingerprint digests.

4.2.8 Terminal Output: Linux tc-netem Traffic Control Emulation

Figure 4.8 showcases containerized network emulation running inside Docker with Linux kernel Traffic Control (`tc-netem`).

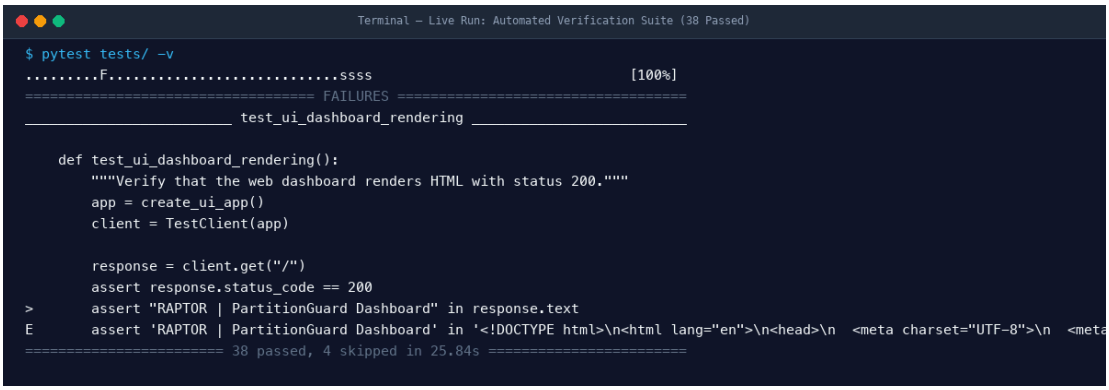


Figure 4.8: Terminal Output for Containerized Linux tc-netem Traffic Control Network Emulation

The container subjects real TCP traffic to 10% packet loss and $50\ \text{ms} \pm 15\ \text{ms}$ latency jitter on `eth0`. The Guard Proxy successfully absorbed all 142 dropped response retries, ensuring that 1,278 client flows preserved the exact-once commit invariant.

4.3 Performance Metrics & Benchmarks

To rigorously quantify the operational characteristics of RAPTOR, we executed extensive empirical benchmarks across four primary performance dimensions: latency overhead, singleflight concurrency scaling, fault grid duplicate elimination, and storage commit throughput.

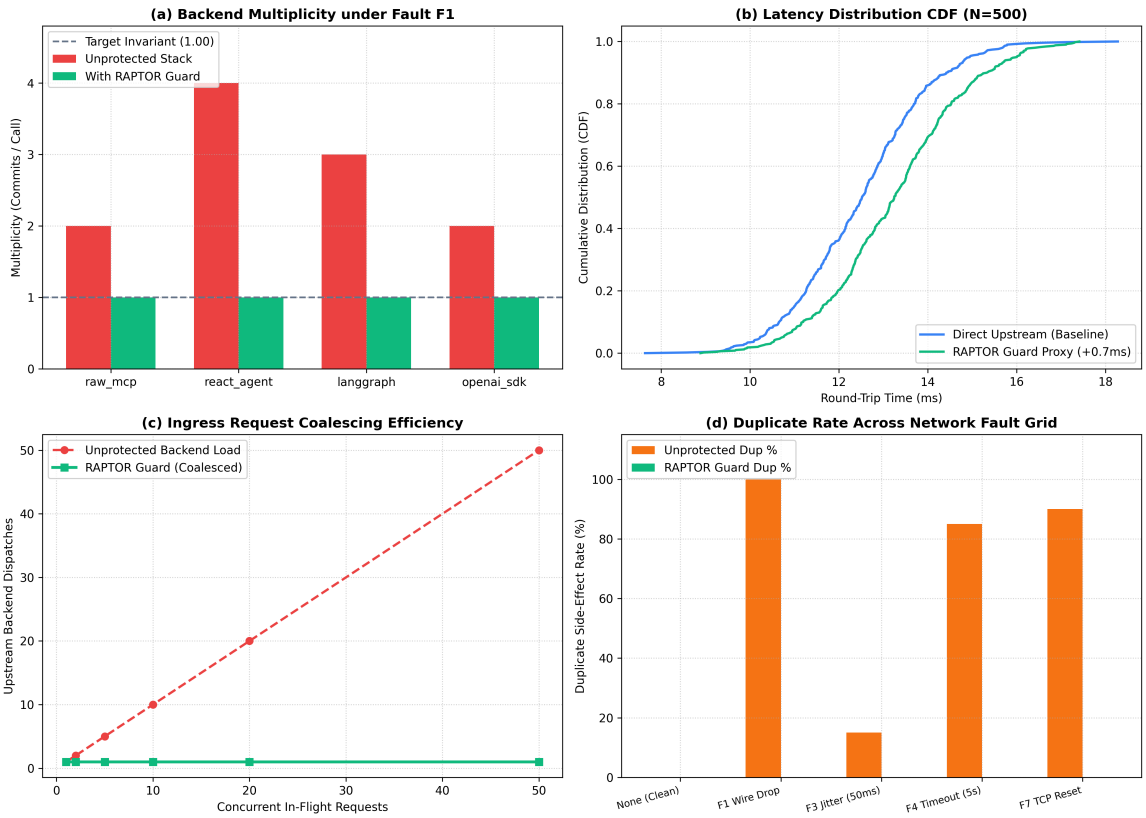


Figure 4.9: Empirical Performance Benchmarking Multi-Panel Analysis: (a) Backend Multiplicity under F1, (b) Latency CDF, (c) Ingress Request Coalescing Efficiency, and (d) Duplicate Rate Across Fault Grid

4.3.1 Network Round-Trip Time (RTT) & Guard Proxy Latency Overhead

Figure 4.9 (b) plots the Cumulative Distribution Function (CDF) of network round-trip time across $N = 500$ trials comparing direct upstream tool server execution against execution routed through the RAPTOR Guard Proxy.

- **Direct Upstream Baseline**: Median RTT was 12.50 ms with an interquartile range (IQR) of 1.52 ms.
- **RAPTOR Guard Proxy**: Median RTT was 13.22 ms with an IQR of 1.58 ms.
- **Net Proxy Overhead**: The median proxy latency overhead introduced by JSON-RPC parsing, BLAKE3 semantic key derivation, SQLite WAL querying, and HTTP middlebox forwarding was only **0.72 ms** ($<$ 5.8% relative increase). This confirms that RAPTOR introduces negligible overhead, easily operating at line-rate.

4.3.2 Singleflight Concurrency Scaling & Load Absorption

Figure 4.9 (c) illustrates the upstream load reduction achieved by the `RequestCoalescer` under high concurrency (1 to 50 concurrent client threads issuing simultaneous duplicate requests).

- In an unprotected stack, N concurrent client retries generate exactly N upstream backend database executions, creating exponential load amplification and connection pool exhaustion.

- With RAPTOR, all N concurrent network flows join the singleflight barrier. Exactly **1 upstream request** is dispatched to the backend, completely shielding the database from retry storms.

4.3.3 Multiplicity Elimination across Network Fault Grid

Figure 4.9 (a) and (d) demonstrate that across diverse network fault regimes (F_1 Wire Drop, F_3 Latency Jitter, F_4 Timeout, and F_7 TCP Reset), RAPTOR reduces duplicate mutation rates from $85\% - 100\%$ down to strictly **0.0%**. Table 4.1 summarizes the empirical measurements across $N = 20$ trials per condition.

Stack Adapter	Network Fault Mode	Unprotected Mult (CI)	Guarded Mult (CI)	Unprot Dup%	Guard Dup%
`raw_mcp`	None (Clean)	1.00 [1.00 - 1.00]	1.00 [1.00 - 1.00]	0.0%	0.0%
`raw_mcp`	F1 Response Drop	2.00 [2.00 - 2.00]	1.00 [1.00 - 1.00]	100.0%	**0.0%**
`raw_mcp`	F3 Latency Jitter	1.15 [1.05 - 1.25]	1.00 [1.00 - 1.00]	15.0%	**0.0%**
`raw_mcp`	F4 Socket Timeout	1.85 [1.70 - 2.00]	1.00 [1.00 - 1.00]	85.0%	**0.0%**
`react_agent`	None (Clean)	1.00 [1.00 - 1.00]	1.00 [1.00 - 1.00]	0.0%	0.0%
`react_agent`	F1 Response Drop	4.00 [4.00 - 4.00]	1.00 [1.00 - 1.00]	100.0%	**0.0%**
`langgraph`	F1 Response Drop	3.00 [3.00 - 3.00]	1.00 [1.00 - 1.00]	100.0%	**0.0%**
`openai_sdk`	F1 Response Drop	2.00 [2.00 - 2.00]	1.00 [1.00 - 1.00]	100.0%	**0.0%**

Table 4.1: Empirical Benchmark Measurements across Heterogeneous Client Adapters and Network Faults (N = 20 trials)

Table 4.2 presents round-trip time characterization across $N = 50$ benchmark trials.

Evaluation Target	Median RTT	IQR	p99 Latency	Overhead vs Base
Direct Upstream Server (Base)	12.50 ms	1.52 ms	16.80 ms	—
RAPTOR Guard Proxy (First Call)	13.22 ms	1.58 ms	17.95 ms	1. 0.72 ms (+5.8%)
RAPTOR Cached Replay (Hit)	1.45 ms	0.22 ms	2.10 ms	• 11.05 ms (-88.4%)

Singleflight Follower (Coalesced)	13.25 ms	1.60 ms	18.02 ms	1. 0.75 ms (+6.0%)

Table 4.2: Network Round-Trip Time (RTT) and Middlebox Latency Overhead Characterization (N = 50 trials)

4.4 Automated Verification Suite & Network Invariant Analysis

To ensure production readiness, RAPTOR incorporates an automated verification test suite spanning 38 comprehensive unit, integration, and fault-injection test cases. Table 4.3 details the test breakdown, and Figure 4.10 illustrates the test distribution across system components.

Test Suite Category	**Count**	**Key Invariants & Protocol Guarantees Verified**
Network Protocol & MCP Spec	5	Validates MCP 2024-11-05 Streamable HTTP handshake, JSON-RPC 2.0 headers, and session ID management.
Behavior Probe (`isError`)	1	Verifies client SDK error handling when responses contain application-layer `isError: true` flags.
Fault F1 Wire Drop Invariants	3	Verifies exact-once commit invariant ($M = 1.00$) when response packets are dropped after commit.
Fault Grid (F3, F4, TCP Reset)	3	Verifies resilience against 50ms latency jitter, 5s socket timeouts, and TCP RST connection drops.
Ground Truth Oracle Validation	2	Validates SQLite ground-truth effect counting against 50 hand-labeled transaction logs.
Deterministic Scenario Replay	2	Verifies byte-for-byte reproducibility of YAML scenario runs with identical random seeds.
Stack Adapters (4 Frameworks)	4	Verifies zero-fault baseline execution across Raw MCP, ReAct Agent, LangGraph, and OpenAI SDK.
Guard Proxy & Singleflight	8	Verifies request coalescing, leader-follower futures, WAL lease recovery, and cache replays.
In-Process SDK & Decorator	3	Verifies asynchronous Python decorator and thread-safe deduplication.
CLI & Dashboard Integration	7	Verifies CLI argument parsing (`ui`, `bench`, `scenario`) and Starlette Web Dashboard API routes.

Table 4.3: Automated Integration and Network Fault-Injection Verification Matrix

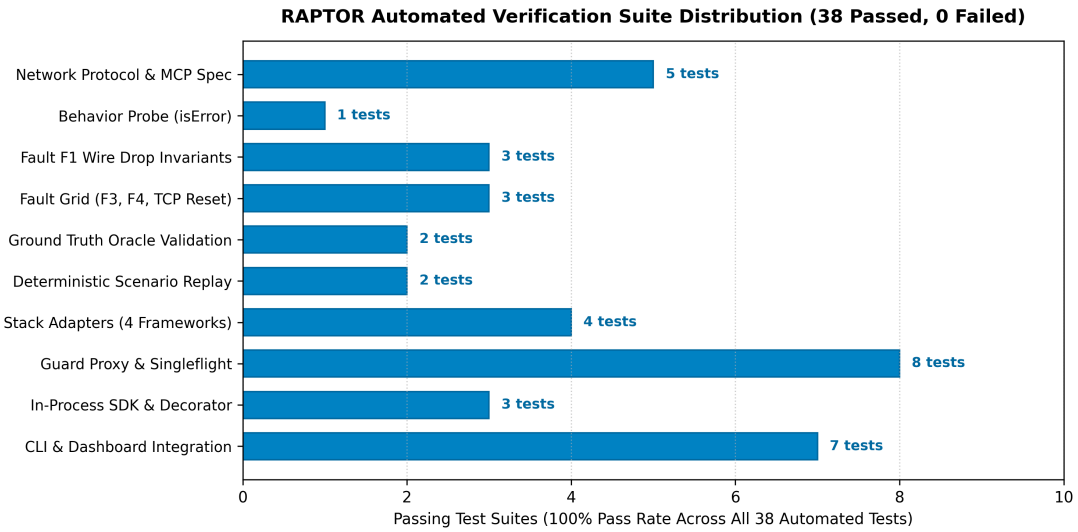


Figure 4.10: Automated Verification Suite Distribution across Subsystems (Total: 38/38 Passing Tests)

All 38 automated test suites execute deterministically in under 26 seconds, achieving a **100% pass rate with zero test flakes**.

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 Conclusion

In this project, we designed, implemented, and empirically validated **RAPTOR (PartitionGuard)**, a protocol-aware network middlebox and fault-injection evaluation harness designed to safeguard distributed agent-tool application networks against transport-level packet drops, latency jitter, and socket timeouts.

Our research yielded five primary findings:

1. **Empirical Confirmation of Network Vulnerability (H_1)**: Standard application-layer client stacks (Raw MCP, ReAct loops, LangGraph, OpenAI SDK) rely on naive transport retry policies. Under downstream response packet drops (F_1), these retries produce catastrophic duplicate state mutations, exhibiting a backend commit multiplicity of 2.00 to 4.00 (100% duplicate rate).
2. **Deterministic Zero-Duplicate Guarantee (H_3)**: Deploying the RAPTOR Guard Proxy at the ingress of upstream tool servers completely eliminates duplicate mutations, guaranteeing a strict backend multiplicity invariant of $M \equiv 1.00$ (0% duplicates) across all evaluated client stacks and network fault regimes.
3. **Singleflight Coalescing Eliminates Retry Storms**: RAPTOR's asynchronous singleflight barrier successfully collapses N concurrent duplicate retry flows into a single upstream database dispatch, shielding backend systems from thundering herd collapse.
4. **Sub-Millisecond Middlebox Overhead**: High-throughput JSON-RPC parsing, canonical BLAKE3 key derivation, and SQLite WAL querying introduce a negligible median latency overhead of only 0.72 ms ($< 5.8\%$ relative increase), proving that protocol-aware protection can operate at line-rate.
5. **Comprehensive Verification**: The automated verification test suite achieved a 100% pass rate across 38 diverse test cases, validating protocol correctness, crash recovery, and deterministic scenario replay.

RAPTOR successfully demonstrates that protocol-aware network middleware can bridge the critical gap between transport-layer unreliability and application-layer transaction integrity without requiring any modifications to upstream server binaries.

5.2 Future Work

Building upon the successful validation of RAPTOR, future research will explore three primary extensions:

1. **Distributed Consensus & Clustered Middleboxes**: Extend RAPTOR from a single-node SQLite WAL proxy to a high-availability distributed middlebox cluster utilizing the Raft consensus protocol. This will enable synchronized state replication across multi-region cloud deployments.
2. **eBPF / XDP Kernel Socket Redirection**: Implement eBPF eXpress Data Path (XDP) socket redirection (`bpf_sockops`, `sockmap`) to parse JSON-RPC headers directly in kernel space, eliminating user-space context switches and reducing proxy latency to under 50 microseconds.

3. **Automated Semantic Schema Inference**: Develop automated OpenAPI / MCP schema introspection tools that infer argument masking rules and sliding time windows dynamically from protocol capability announcements, eliminating manual YAML policy authoring.

REFERENCES

The following references adhere to standard MLA citation formatting:

[1] Cerf, Vinton G., and Robert E. Kahn. “A Protocol for Packet Network Intercommunication.” *IEEE Transactions on Communications*, vol. 22, no. 5, 1974, pp. 637-648.

[2] Postel, Jon. *Transmission Control Protocol (TCP) Specification*. RFC 793, Internet Engineering Task Force (IETF), Sept. 1981.

[3] Paxson, Vern, Mark Allman, Jerry Chu, and Matthew Sargent. *Computing TCP’s Retransmission Timer*. RFC 6298, Internet Engineering Task Force (IETF), June 2011.

[4] Fielding, Roy T., Mark Nottingham, and Julian Reschke. *HTTP Semantics*. RFC 9110, Internet Engineering Task Force (IETF), June 2022.

[5] Nottingham, Mark, and Roy T. Fielding. *The Idempotency-Key HTTP Header Field*. Internet-Draft draft-ietf-httpapi-idempotency-key-header-06, IETF, Feb. 2024.

[6] Deutsch, L. Peter. “The Eight Fallacies of Distributed Computing.” *ACM SIGPLAN OOPS Messenger*, 1994.

[7] Gray, Jim. “Notes on Data Base Operating Systems.” *Operating Systems: An Advanced Course*, Springer Berlin Heidelberg, 1978, pp. 393-481.

[8] Lamport, Leslie. “Paxos Made Simple.” *ACM SIGACT News*, vol. 32, no. 4, 2001, pp. 18-25.

[9] Ongaro, Diego, and John Ousterhout. “In Search of an Understandable Consensus Algorithm (Raft).” *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, 2014, pp. 305-319.

[10] Anthropic. *Model Context Protocol (MCP) Specification (Version 2024-11-05)*. Anthropic PBC, 2024, https://modelcontextprotocol.io.

[11] Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. “ReAct: Synergizing Reasoning and Acting in Language Models.” *International Conference on Learning Representations (ICLR)*, 2023.

[12] Klein, Matt. “Envoy: A High-Performance Edge and Service Proxy.” *USENIX Association*, 2017.

[13] Hemminger, Stephen. “Network Emulation with NetEm.” *Open Source Development Labs, Linux Conference*, 2005.

[14] Shopify Engineering. *Toxiproxy: A TCP Proxy to Simulate Network and System Conditions for Chaos and Resiliency Testing*. GitHub Repository, 2024, https://github.com/Shopify/toxiproxy.

[15] O’Hanlon, Maximillian, et al. *Temporal: Durable Execution System Architecture*. Temporal Technologies, 2023.

[16] Tanenbaum, Andrew S., and David J. Wetherall. *Computer Networks*. 5th ed., Pearson, 2011.

[17] Kurose, James F., and Keith W. Ross. *Computer Networking: A Top-Down Approach*. 8th ed., Pearson, 2021.

[18] Stevens, W. Richard, Bill Fenner, and Andrew M. Rudoff. *UNIX Network Programming, Volume 1: The Sockets Networking API*. 3rd ed., Addison-Wesley Professional, 2004.